

Assignment 4

TITLE: VECTOR AND MATRIX OPERATIONS.

PROBLEM STATEMENT: Write a CUDA Program for:

1. Addition of two large vectors
2. Matrix Multiplication using CUDA

PRE-REQUISITES:

Knowledge of dividing a large problem into sub-problems. Identify data dependency among them. Knowledge of basic matrix and vector operations.

Steps for Addition of two large vectors using CUDA

1. Define the size of the vectors: In this step, you need to define the size of the vectors that you want to add. This will determine the number of threads and blocks you will need to use to parallelize the addition operation.
2. Allocate memory on the host: In this step, you need to allocate memory on the host for the two vectors that you want to add and for the result vector. You can use the C malloc function to allocate memory.
3. Initialize the vectors: In this step, you need to initialize the two vectors that you want to add on the host. You can use a loop to fill the vectors with data.
4. Allocate memory on the device: In this step, you need to allocate memory on the device for the two vectors that you want to add and for the result vector. You can use the CUDA function cudaMalloc to allocate memory.
5. Copy the input vectors from host to device: In this step, you need to copy the two input vectors from the host to the device memory. You can use the CUDA function cudaMemcpy to copy the vectors.
6. Launch the kernel: In this step, you need to launch the CUDA kernel that will perform the addition operation. The kernel will be executed by multiple threads in parallel. You can use the <<<...>>> syntax to specify the number of blocks and threads to use.

7. Copy the result vector from device to host: In this step, you need to copy the result vector from the device memory to the host memory. You can use the CUDA function `cudaMemcpy` to copy the result vector.
8. Free memory on the device: In this step, you need to free the memory that was allocated on the device. You can use the CUDA function `cudaFree` to free the memory.
9. Free memory on the host: In this step, you need to free the memory that was allocated on the host. You can use the C free function to free the memory.

Steps for Matrix Multiplication using CUDA

Here are the steps for implementing matrix multiplication using CUDA C:

1. Matrix Initialization: The first step is to initialize the matrices that you want to multiply. You can use standard C or CUDA functions to allocate memory for the matrices and initialize their values. The matrices are usually represented as 2D arrays.
2. Memory Allocation: The next step is to allocate memory on the host and the device for the matrices. You can use the standard C `malloc` function to allocate memory on the host and the CUDA function `cudaMalloc()` to allocate memory on the device.
3. Data Transfer: The third step is to transfer data between the host and the device. You can use the CUDA function `cudaMemcpy()` to transfer data from the host to the device or vice versa.
4. Kernel Launch: The fourth step is to launch the CUDA kernel that will perform the matrix multiplication on the device. You can use the `<<<...>>>` syntax to specify the number of blocks and threads to use. Each thread in the kernel will compute one element of the output matrix.
5. Device Synchronization: The fifth step is to synchronize the device to ensure that all kernel executions have completed before proceeding. You can use the CUDA function `cudaDeviceSynchronize()` to synchronize the device.
6. Data Retrieval: The sixth step is to retrieve the result of the computation from the device to the host. You can use the CUDA function `cudaMemcpy()` to transfer data from the device to the host.

7. **Memory Deallocation:** The final step is to deallocate the memory that was allocated on the host and the device. You can use the C free function to deallocate memory on the host and the CUDA function `cudaFree()` to deallocate memory on the device.

Conclusion:

CUDA enables faster and more efficient execution of large vector and matrix operations by utilizing parallel processing on the GPU, making it ideal for high-performance computing tasks.



```
In [ ]: # Steps to Setup Collab NoteBook for CUDA Programming

# In Menu Bar Click on Runtime -> Change Runtime type -> Select T4 GPU

# Then execute the below command to get real-time information about your GPU a
```

```
In [ ]: !nvidia-smi
```

```
Wed Mar 25 05:44:27 2026
+-----+
+-----+
| NVIDIA-SMI 580.82.07                Driver Version: 580.82.07      CUDA Versio
n: 13.0      |
+-----+-----+
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile U
ncorr. ECC |
| Fan   Temp   Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M. |
| MIG M. |
|=====+=====+
| 0 Tesla T4                        Off | 00000000:00:04.0 Off |
0 |
| N/A    45C    P8              13W / 70W |      0MiB / 15360MiB |      0%
Default |
|
| N/A |
+-----+-----+
+-----+-----+
+-----+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name
GPU Memory |
|      ID    ID
Usage      |
|=====+=====+
|=====+=====+
| No running processes found
|
+-----+-----+
+-----+-----+
```

```
In [ ]: # Below cell contains the code and the command to write the program to your CL

# '%writefile nspdp.cu' this command has keyword 'writefile' which means the

# The command is not necessary you can create a seperate file with .cu extensi
```

In [34]: %%writefile vector_add.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define N 10000000 // Large vector size
#define DISPLAY_N 5 // Elements to show at start and end

__global__ void vectorAdd(float *A, float *B, float *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Grid-stride loop
    while (i < n) {
        C[i] = A[i] + B[i];
        i += blockDim.x * gridDim.x;
    }
}

int main() {
    float *h_A, *h_B, *h_C;
    float *d_A, *d_B, *d_C;

    size_t size = N * sizeof(float);

    // Allocate memory
    h_A = (float*)malloc(size);
    h_B = (float*)malloc(size);
    h_C = (float*)malloc(size);

    // Initialize vectors
    for (int i = 0; i < N; i++) {
        h_A[i] = (i % 5) + 1; // Pattern 1-5
        h_B[i] = (i % 3) + 1; // Pattern 1-3
    }

    // ===== VECTOR A =====
    printf("\nVector A (%d elements):\n", N);

    for (int i = 0; i < DISPLAY_N; i++)
        printf("%.0f ", h_A[i]);

    printf("... ");

    for (int i = N - DISPLAY_N; i < N; i++)
        printf("%.0f ", h_A[i]);

    printf("\n");

    // ===== VECTOR B =====
    printf("\nVector B (%d elements):\n", N);

    for (int i = 0; i < DISPLAY_N; i++)
        printf("%.0f ", h_B[i]);
```

```

printf("... ");

for (int i = N - DISPLAY_N; i < N; i++)
    printf("%.0f ", h_B[i]);

printf("\n");

// Allocate device memory
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

// Copy to GPU
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int blockSize = 256;
int gridSize = 1024;

// Launch kernel
vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, N);

// Copy result back
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// ===== VECTOR C =====
printf("\nVector C (%d elements):\n", N);

for (int i = 0; i < DISPLAY_N; i++)
    printf("%.0f ", h_C[i]);

printf("... ");

for (int i = N - DISPLAY_N; i < N; i++)
    printf("%.0f ", h_C[i]);

printf("\n");

// Free memory
free(h_A); free(h_B); free(h_C);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);

return 0;
}

```

Overwriting vector_add.cu

```

In [ ]: %%writefile matrix_mul.cu
#include <stdio.h>
#include <cuda_runtime.h>

#define N 1024          // Large computation size
#define DISPLAY_N 4     // Rows/columns to display

```

```

// CUDA Kernel
__global__ void matrixMul(float *A, float *B, float *C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n && col < n) {
        float sum = 0.0f;

        for (int k = 0; k < n; k++) {
            sum += A[row * n + k] * B[k * n + col];
        }

        C[row * n + col] = sum;
    }
}

int main() {
    float *h_A, *h_B, *h_C;
    float *d_A, *d_B, *d_C;

    size_t size = N * N * sizeof(float);

    // Allocate host memory
    h_A = (float*)malloc(size);
    h_B = (float*)malloc(size);
    h_C = (float*)malloc(size);

    // Initialize matrices
    for (int i = 0; i < N * N; i++) {
        h_A[i] = (i % 5) + 1; // Values 1-5
        h_B[i] = (i % 3) + 1; // Values 1-3
    }

    // ===== MATRIX A =====
    printf("\nMatrix A (%dx%d):\n", N, N);

    for (int i = 0; i < DISPLAY_N; i++) {
        for (int j = 0; j < DISPLAY_N; j++)
            printf("%.0f ", h_A[i*N + j]);
        printf("... %.0f\n", h_A[i*N + (N - 1)]);
    }

    printf("...\n");

    for (int i = N - DISPLAY_N; i < N; i++) {
        for (int j = 0; j < DISPLAY_N; j++)
            printf("%.0f ", h_A[i*N + j]);
        printf("... %.0f\n", h_A[i*N + (N - 1)]);
    }

    // ===== MATRIX B =====
    printf("\nMatrix B (%dx%d):\n", N, N);

```

```

for (int i = 0; i < DISPLAY_N; i++) {
    for (int j = 0; j < DISPLAY_N; j++)
        printf("%.0f ", h_B[i*N + j]);
    printf("... %.0f\n", h_B[i*N + (N - 1)]);
}

printf("...\n");

for (int i = N - DISPLAY_N; i < N; i++) {
    for (int j = 0; j < DISPLAY_N; j++)
        printf("%.0f ", h_B[i*N + j]);
    printf("... %.0f\n", h_B[i*N + (N - 1)]);
}

// Allocate device memory
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

// Copy data to GPU
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

// Kernel launch
dim3 blockSize(16, 16);
dim3 gridSize((N + 15) / 16, (N + 15) / 16);

matrixMul<<<gridSize, blockSize>>>(d_A, d_B, d_C, N);

// Copy result back
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// ===== MATRIX C =====
printf("\nMatrix C (%dx%d):\n", N, N);

for (int i = 0; i < DISPLAY_N; i++) {
    for (int j = 0; j < DISPLAY_N; j++)
        printf("%.0f ", h_C[i*N + j]);
    printf("... %.0f\n", h_C[i*N + (N - 1)]);
}

printf("...\n");

for (int i = N - DISPLAY_N; i < N; i++) {
    for (int j = 0; j < DISPLAY_N; j++)
        printf("%.0f ", h_C[i*N + j]);
    printf("... %.0f\n", h_C[i*N + (N - 1)]);
}

// Free memory
free(h_A); free(h_B); free(h_C);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);

return 0;

```



```
}
```

Overwriting matrix_mul.cu

```
In [ ]: # The below command is used to compile the CUDA Program into executable file  
# nvcc stands for NVIDIA CUDA Compiler.  
# It is a compiler for GPU Programs and compiles .cu files into executable file
```

```
In [35]: !nvcc vector_add.cu -o vector_add
```

nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (Use -Wno-deprecated-gpu-targets to suppress warning).

```
In [ ]: !nvcc matrix_mul.cu -o matrix_mul
```

nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (Use -Wno-deprecated-gpu-targets to suppress warning).

```
In [ ]: # The below command is used to execute the executable file and fetch the output
```

```
In [36]: !./vector_add
```

Vector A (10000000 elements):

1 2 3 4 5 ... 1 2 3 4 5

Vector B (10000000 elements):

1 2 3 1 2 ... 3 1 2 3 1

Vector C (10000000 elements):

2 4 6 5 7 ... 4 3 5 7 6

```
In [ ]: !./matrix_mul
```

Matrix A (1024x1024):

```
1 2 3 4 ... 4
5 1 2 3 ... 3
4 5 1 2 ... 2
3 4 5 1 ... 1
...
1 2 3 4 ... 4
5 1 2 3 ... 3
4 5 1 2 ... 2
3 4 5 1 ... 1
```

Matrix B (1024x1024):

```
1 2 3 1 ... 1
2 3 1 2 ... 2
3 1 2 3 ... 3
1 2 3 1 ... 1
...
1 2 3 1 ... 1
2 3 1 2 ... 2
3 1 2 3 ... 3
1 2 3 1 ... 1
```

Matrix C (1024x1024):

```
6138 6139 6143 6138 ... 6138
6136 6141 6149 6136 ... 6136
6139 6148 6145 6139 ... 6139
6147 6145 6146 6147 ... 6147
...
6138 6139 6143 6138 ... 6138
6136 6141 6149 6136 ... 6136
6139 6148 6145 6139 ... 6139
6147 6145 6146 6147 ... 6147
```